

1. 校准文件说明

「 1.1. 校准文件如何获取 」

D:\Nano\Cuga-Calibration\CalibrationResult\Result_日期时间.dat

- 按照日期进行分类排序存储的

「 1.2. 怎么验证校准对象是否可以使用 」

任何校准对象都是继承自 `CalibrationBase` , 所以需要使用 `CalibrationBase.IsOk` 判断改校准对象是否能够使用

```
1 [Serializable]
2 public class CalibrationBase
3 {
4     /// <summary>
5     /// 是否校准Ok
6     /// </summary>
7     public bool IsCalibrated { get; set; }
8
9     /// <summary>
10    /// 是否验证Ok
11    /// </summary>
12    public bool IsVerified { get; set; }
13
14    /// <summary>
15    /// 是否Ok
16    /// </summary>
17    [JsonIgnore]
18    public bool IsOk => IsCalibrated && IsVerified;
19 }
```

「 1.3. 有哪些大类 」

一共4大类

ADS: `CalibrationObj.CalibrationAdsObj`

Chuck: `CalibrationObj.CalibrationChuckObj`

激光: `CalibrationObj.CalibrationLaserObj`

显微镜: `CalibrationObj.CalibrationMicroscopeObj`

```
1 /// <summary>
2 /// 校准对象序列化
```

```

3  /// </summary>
4  [Serializable]
5  public sealed class CalibrationObj
6  {
7      /// <summary>
8      /// 缓震平台校准对象
9      /// </summary>
10     public CalibrationAdsObj CalibrationAdsObj { get; set; } = new
    CalibrationAdsObj();
11
12     /// <summary>
13     /// Chuck校准对象
14     /// </summary>
15     public CalibrationChuckObj CalibrationChuckObj { get; set; } = new
    CalibrationChuckObj();
16
17     /// <summary>
18     /// 激光校准对象
19     /// </summary>
20     public CalibrationLaserObj CalibrationLaserObj { get; set; } = new
    CalibrationLaserObj();
21
22     /// <summary>
23     /// 显微镜校准对象
24     /// </summary>
25     public CalibrationMicroscopeObj CalibrationMicroscopeObj { get; set; }
    = new CalibrationMicroscopeObj();
26 }

```

2. 显微镜: CalibrationMicroscopeObj

显微镜一共4个校准: 自动聚焦校准、Cal Chip4个耳朵校准、尺寸校准、中心校准: 分别对应一下属性

自动聚焦校准: `CalibrationMicroscopeObj.CalibrationMicroscopeFocusItemList` 是列表, 列表中有5个对象, 因为我们有多个镜头5X 10X 50X 100X 150X

Cal Chip4个耳朵校准: `CalibrationMicroscopeObj.CalibrationMicroscopeCalChip`

尺寸校准: `CalibrationMicroscopeObj.CalibrationMicroscopePixelSizeItemList` 是列表, 列表中有5个对象, 因为我们有多个镜头5X 10X 50X 100X 150X

中心校准: `CalibrationMicroscopeObj.CalibrationMicroscopeCentricityItemList` 是列表, 列表中有5个对象, 因为我们有多个镜头5X 10X 50X 100X 150X

```

1  /// <summary>
2  /// 显微镜校准对象
3  /// </summary>
4  [Serializable]
5  public sealed class CalibrationMicroscopeObj
6  {
7      /// <summary>
8      /// Focus校准对象列表

```

```

9      /// </summary>
10     public List<CalibrationMicroscopeFocusItem>
CalibrationMicroscopeFocusItemList { get; set; } = new
List<CalibrationMicroscopeFocusItem>(0);
11
12     /// <summary>
13     /// Cal Chip校准对象
14     /// </summary>
15     public CalibrationMicroscopeCalChip CalibrationMicroscopeCalChip { get;
set; } = new CalibrationMicroscopeCalChip();
16
17     /// <summary>
18     /// PixelSize校准对象列表
19     /// </summary>
20     public List<CalibrationMicroscopePixelSizeItem>
CalibrationMicroscopePixelSizeItemList { get; set; } = new
List<CalibrationMicroscopePixelSizeItem>(0);
21
22     /// <summary>
23     /// Centricity校准对象列表
24     /// </summary>
25     public List<CalibrationMicroscopeCentricityItem>
CalibrationMicroscopeCentricityItemList { get; set; } = new
List<CalibrationMicroscopeCentricityItem>(0);
26 }

```

「 2.1. 自动聚焦校准： CalibrationMicroscopeFocusItem 」

需要使用的属性为一下3个：

CalibrationMicroscopeFocusItem.CgMicroscopeLens ：当前镜头（cuga中显微镜枚举类型，分别对应了5个镜头）

CalibrationMicroscopeFocusItem.EcsValue ：当前镜头的最佳清晰高度（绝对ECS），**需要下发AF硬件**

```

1 public SxExecuteRet<bool>
SetSensorStandardEcsValue(MicroscopeMagnificationEnum
microscopeMagnificationEnum, double standardEcsValue)
2 {
3     return SxExecuteRetHelper.CreateSuccess(true);
4     throw new NotImplementedException();
5 }

```

CalibrationMicroscopeFocusItem.MicroscopeVoltage ：当前镜头的的显微镜相像差镜头电压值（绝对电压），**需要下发Microscope硬件**

```

1 public SxExecuteRet<bool> SetVoltage(double voltage)
2 {
3     var sxExecuteRet = Invoke(() =>
4         Service!.WriteMicroscopeVolt(Convert.ToInt32(voltage)));
5     return sxExecuteRet.IsSuccess == false
6         ? SxExecuteRetHelper.CreateError(sxExecuteRet.Msg, false)
7         : SxExecuteRetHelper.CreateSuccess(true);
8 }

```

```

1 /// <summary>
2 /// Focus校准对象
3 /// </summary>
4 [Serializable]
5 public sealed class CalibrationMicroscopeFocusItem : CalibrationBase
6 {
7     /// <summary>
8     /// 显微镜镜头
9     /// </summary>
10    public CgMicroscopeLens CgMicroscopeLens { get; set; }
11
12    /// <summary>
13    /// Ecs值
14    /// </summary>
15    public double EcsValue { get; set; }
16
17    /// <summary>
18    /// 显微镜电压值(像差镜头绝对位置)
19    /// </summary>
20    public double MicroscopeVoltage { get; set; }
21 }

```

「 2.2. Cal Chip4个耳朵校准: CalibrationMicroscopeCalChip 」

需要使用的属性为一下4个:

CalChipSiteModelEnum.ChuckModel => 0,

CalChipSiteModelEnum.UndefinedModel => 1,

CalChipSiteModelEnum.DswModel => 2,

ChipSiteModelEnum.ShinyWaferModel => 3,

ChipSiteModelEnum.HazeModel => 4,

CalibrationMicroscopeCalChip.DswBrightFieldMachinePosition : DSW 明场中心的机械位置 (绝对位置), 需要下发AF硬件

CalibrationMicroscopeCalChip.DswDarkFieldMachinePosition : DSW 暗场中心的机械位置 (绝对位置), 需要下发AF硬件

CalibrationMicroscopeCalChip.DswDarkFieldMachinePosition : DSW 5X的最佳清晰高度 (绝对ECS), 需要下发AF硬件

`CalibrationMicroscopeCalChip.UndefinedBrightFieldMachinePosition` : Undefined
明场中心的机械位置 (绝对位置), 需要下发AF硬件

`CalibrationMicroscopeCalChip.UndefinedDarkFieldMachinePosition` : Undefined
暗场中心的机械位置 (绝对位置), 需要下发AF硬件

`CalibrationMicroscopeCalChip.UndefinedDarkFieldMachinePosition` : Undefined
5X的最佳清晰高度 (绝对ECS), 需要下发AF硬件

`CalibrationMicroscopeCalChip.HazeBrightFieldMachinePosition` : Haze
明场中心的机械位置 (绝对位置), 需要下发AF硬件

`CalibrationMicroscopeCalChip.HazeDarkFieldMachinePosition` : Haze
暗场中心的机械位置 (绝对位置), 需要下发AF硬件

`CalibrationMicroscopeCalChip.HazeDarkFieldMachinePosition` : Haze
5X的最佳清晰高度 (绝对ECS), 需要下发AF硬件

`CalibrationMicroscopeCalChip.ShinyWaferBrightFieldMachinePosition` :
ShinyWafer 明场中心的机械位置 (绝对位置), 需要下发AF硬件

`CalibrationMicroscopeCalChip.ShinyWaferDarkFieldMachinePosition` : ShinyWafer
暗场中心的机械位置 (绝对位置), 需要下发AF硬件

`CalibrationMicroscopeCalChip.ShinyWaferDarkFieldMachinePosition` : ShinyWafer
5X的最佳清晰高度 (绝对ECS), 需要下发AF硬件

```
1 public SxExecuteRet<bool>
   SetSensorBrightFieldCalChipCenterMachinePositionValue(CalChipSiteModel
   Enum calChipSiteModelEnum, Point position)
2 {
3     switch (calChipSiteModelEnum.ToCgCalChipModel())
4     {
5         case 1:
6             var sxExecuteRet1 = Invoke(() =>
7 Service!.SetAFBFCalChipLUloaction(position.ToCgPoint()));
8             return sxExecuteRet1.IsSuccess == false
9                 ? SxExecuteRetHelper.CreateError(sxExecuteRet1.Msg,
10 false)
11                 : SxExecuteRetHelper.CreateSuccess(true);
12         case 2:
13             var sxExecuteRet2 = Invoke(() =>
14 Service!.SetAFBFCalChipRUloaction(position.ToCgPoint()));
15             return sxExecuteRet2.IsSuccess == false
16                 ? SxExecuteRetHelper.CreateError(sxExecuteRet2.Msg,
17 false)
18                 : SxExecuteRetHelper.CreateSuccess(true);
19         case 3:
20             var sxExecuteRet3 = Invoke(() =>
21 Service!.SetAFBFCalChipRDloaction(position.ToCgPoint()));
22             return sxExecuteRet3.IsSuccess == false
23                 ? SxExecuteRetHelper.CreateError(sxExecuteRet3.Msg,
24 false)
25                 : SxExecuteRetHelper.CreateSuccess(true);
```

```

26         case 4:
27             var sxExecuteRet4 = Invoke(() =>
Service!.SetAFBFCalChipLDLoaction(position.ToCgPoint()));
28
29             return sxExecuteRet4.IsSuccess == false
30                 ? SxExecuteRetHelper.CreateError(sxExecuteRet4.Msg,
false)
31                 : SxExecuteRetHelper.CreateSuccess(true);
32
33         default:
34             throw new
ArgumentOutOfRangeException(nameof(calChipSiteModelEnum),
calChipSiteModelEnum, null);
35     }
36 }
37
38 public SxExecuteRet<bool>
SetSensorBrightFieldCalChipStandardEcsValue(CalChipSiteModelEnum
calChipSiteModelEnum, double standardEcsValue)
39 {
40     switch (calChipSiteModelEnum.ToCgCalChipModel())
41     {
42         case 1:
43             var sxExecuteRet1 = Invoke(() =>
Service!.SetAFBFCalChipLUHeight(Convert.ToUInt16(standardEcsValue)));
44
45             return sxExecuteRet1.IsSuccess == false
46                 ? SxExecuteRetHelper.CreateError(sxExecuteRet1.Msg,
false)
47                 : SxExecuteRetHelper.CreateSuccess(true);
48
49         case 2:
50             var sxExecuteRet2 = Invoke(() =>
Service!.SetAFBFCalChipRUHeight(Convert.ToUInt16(standardEcsValue)));
51
52             return sxExecuteRet2.IsSuccess == false
53                 ? SxExecuteRetHelper.CreateError(sxExecuteRet2.Msg,
false)
54                 : SxExecuteRetHelper.CreateSuccess(true);
55
56         case 3:
57             var sxExecuteRet3 = Invoke(() =>
Service!.SetAFBFCalChipRDHeight(Convert.ToUInt16(standardEcsValue)));
58
59             return sxExecuteRet3.IsSuccess == false
60                 ? SxExecuteRetHelper.CreateError(sxExecuteRet3.Msg,
false)
61                 : SxExecuteRetHelper.CreateSuccess(true);
62
63         case 4:
64             var sxExecuteRet4 = Invoke(() =>
Service!.SetAFBFCalChipLDHeight(Convert.ToUInt16(standardEcsValue)));
65
66             return sxExecuteRet4.IsSuccess == false

```

```

67         ? SxExecuteRetHelper.CreateError(sxExecuteRet4.Msg,
        false)
68         : SxExecuteRetHelper.CreateSuccess(true);
69
70     default:
71         throw new
        ArgumentOutOfRangeException(nameof(calChipSiteModelEnum),
        calChipSiteModelEnum, null);
72     }
73 }

```

```

1  /// <summary>
2  /// Cal Chip校准对象
3  /// </summary>
4  [Serializable]
5  public sealed class CalibrationMicroscopeCalChip : CalibrationBase
6  {
7      /// <summary>
8      /// Dsw明场机械位置
9      /// </summary>
10     public CgPoint DswBrightFieldMachinePosition { get; set; }
11
12     /// <summary>
13     /// Dsw暗场机械位置
14     /// </summary>
15     public CgPoint DswDarkFieldMachinePosition { get; set; }
16
17     /// <summary>
18     /// DswEcs值
19     /// </summary>
20     public double DswEcsValue { get; set; }
21
22     /// <summary>
23     /// Undefined明场机械位置
24     /// </summary>
25     public CgPoint UndefinedBrightFieldMachinePosition { get; set; }
26
27     /// <summary>
28     /// Undefined暗场机械位置
29     /// </summary>
30     public CgPoint UndefinedDarkFieldMachinePosition { get; set; }
31
32     /// <summary>
33     /// UndefinedEcs值
34     /// </summary>
35     public double UndefinedEcsValue { get; set; }
36
37     /// <summary>
38     /// Haze明场机械位置
39     /// </summary>
40     public CgPoint HazeBrightFieldMachinePosition { get; set; }
41

```

```

42     /// <summary>
43     /// Haze暗场机械位置
44     /// </summary>
45     public CgPoint HazeDarkFieldMachinePosition { get; set; }
46
47     /// <summary>
48     /// HazeEcs值
49     /// </summary>
50     public double HazeEcsValue { get; set; }
51
52     /// <summary>
53     /// Shiny Wafer明场机械位置
54     /// </summary>
55     public CgPoint ShinyWaferBrightFieldMachinePosition { get; set; }
56
57     /// <summary>
58     /// Shiny Wafer暗场机械位置
59     /// </summary>
60     public CgPoint ShinyWaferDarkFieldMachinePosition { get; set; }
61
62     /// <summary>
63     /// ShinyWaferEcs值
64     /// </summary>
65     public double ShinyWaferEcsValue { get; set; }
66 }

```

「 2.3. 尺寸校准： CalibrationMicroscopePixelSizeItem 」

需要使用的属性为一下2个：

CalibrationMicroscopePixelSizeItem.CgMicroscopeLens ：当前镜头（cuga中显微镜枚举类型，分别对应了5个镜头）

CalibrationMicroscopePixelSizeItem.PixelSize ：当前镜头的x方向和y方向1像素转尺寸（um），**Cuga内部使用**

```

1  /// <summary>
2  /// PixelSize校准对象
3  /// </summary>
4  [Serializable]
5  public sealed class CalibrationMicroscopePixelSizeItem : CalibrationBase
6  {
7      /// <summary>
8      /// 显微镜镜头
9      /// </summary>
10     public CgMicroscopeLens CgMicroscopeLens { get; set; }
11
12     /// <summary>
13     /// PixelSize, 单位um/pixel(SizePerPixel)
14     /// </summary>
15     public CgSize PixelSize { get; set; }

```


「 2.4. 中心校准： CalibrationMicroscopeCentricityItem 」

需要使用的属性为一下2个：

`CalibrationMicroscopeCentricityItem.CgMicroscopeLens`：当前镜头（cuga中显微镜枚举类型，分别对应了5个镜头）

`CalibrationMicroscopeCentricityItem.Offset`：当前镜头的中心位置偏差【相对距离】（都是和150X偏差，所以可以通过这个可以计算任意两个镜头差），**Cuga内部使用**

```

1  /// <summary>
2  /// Centricity校准对象
3  /// </summary>
4  [Serializable]
5  public sealed class CalibrationMicroscopeCentricityItem : CalibrationBase
6  {
7      /// <summary>
8      /// 显微镜镜头
9      /// </summary>
10     public CgMicroscopeLens CgMicroscopeLens { get; set; }
11
12     /// <summary>
13     /// 显微镜中心镜头位置偏差(当前镜头 - 150X镜头)
14     /// </summary>
15     public CgPoint Offset { get; set; }
16 }
```

3. Chuck: CalibrationChuckObj

Chuck一共5个校准：分别对应以下属性

Prealigner校准：`CalibrationPrealignerObj`

正交校准：`CalibrationChuckObj.CalibrationChuckGantry`

Chuck Center校准：`CalibrationCenterObj`

```

1  /// <summary>
2  /// Chuck校准对象
3  /// </summary>
4  [Serializable]
5  public sealed class CalibrationChuckObj
6  {
7      /// <summary>
8      /// Prealigner校准对象
9      /// </summary>
10     public CalibrationPrealignerObj CalibrationPrealignerObj { get; set; }
11     = new CalibrationPrealignerObj();
```

```

12     /// <summary>
13     /// Gantry正交校准对象
14     /// </summary>
15     public CalibrationChuckGantry CalibrationChuckGantry { get; set; } =
new CalibrationChuckGantry();
16
17     /// <summary>
18     /// Chuck Center校准对象
19     /// </summary>
20     public CalibrationCenterObj CalibrationCenterObj { get; set; } = new
CalibrationCenterObj();
21 }

```

「 3.1.Prealigner校准: CalibrationPrealignerObj 」

需要使用以下属性:

CalibrationPrealignerObj.NewEfemLoadWaferStagePosition : 校准后EFEM上下料位置的Stage物理坐标 (校准时EFEM上下料位置的Stage物理坐标+校准得出的晶圆中心偏移量)

CalibrationPrealignerObj.NewEfemLoadWaferChuckAngle : 校准后EFEM上下料时Chuck的起始旋转角度 (校准时EFEM上下料时Chuck的起始旋转角度+校准得出的晶圆偏移角度)

```

1     public sealed class CalibrationPrealignerObj : CalibrationBase
2     {
3         /// <summary>
4         /// 校准后EFEM上下料位置的Stage物理坐标
5         /// </summary>
6         public CgPoint NewEfemLoadWaferStagePosition { get; set; }
7
8         /// <summary>
9         /// 校准后EFEM上下料时Chuck的起始旋转角度
10        /// </summary>
11        public double NewEfemLoadWaferChuckAngle { get; set; }
12    }
13
14

```

「 3.2. 正交校准: CalibrationChuckGantry 」

需要使用的属性为以下1个:

CalibrationChuckGantry.Offset : Y轴正交误差 (绝对偏差) , 需要下发ACS硬件

```

1 public SxExecuteRet<bool> SetGantryOffset(double gantryOffset)
2 {
3     var sxExecuteRet = Invoke(() => Service!.SetGantry(gantryOffset));
4
5     return sxExecuteRet.IsSuccess == false
6         ? SxExecuteRetHelper.CreateError(sxExecuteRet.Msg, false)
7         : SxExecuteRetHelper.CreateSuccess(true);
8 }

```

```

1 /// <summary>
2 /// Gantry正交校准对象
3 /// </summary>
4 public sealed class CalibrationChuckGantry : CalibrationBase
5 {
6     /// <summary>
7     /// 整个Y轴的误差
8     /// </summary>
9     public double Offset { get; set; }
10 }

```

「 3.3.Chuck Center校准: CalibrationCenterObj 」

需要使用以下属性:

CalibrationCenterObj.NewBFCenterStagePosition : 校准后明场中心Stage的物理坐标 (校准时明场中心Stage的物理坐标+校准后Chuck Center中心偏移量)

```

1 public sealed class CalibrationCenterObj : CalibrationBase
2 {
3     /// <summary>
4     /// 校准后明场中心Stage的物理坐标
5     /// </summary>
6     public CgPoint NewBFCenterStagePosition { get; set; }
7 }

```

4. 激光: CalibrationLaserObj

Laser一共7个校准:

暗场自动聚焦、Aod延迟校准、暗场相机尺寸、暗场自动聚焦: 分别对应一下属性

自动聚焦校准: **CalibrationLaserObj.CalibrationLaserAutoFocus**

Aod延迟校准: **CalibrationLaserObj.CalibrationLaserAodDelay**

暗场相机尺寸校准: **CalibrationLaserObj.CalibrationLaserPixelSizeItemList** 是列表, 列表中有3*15个对象, 分别对用有3个mag (低中高) * 15个通道3PMT暗场相机 (注意: 我们硬件目前只有高Mag, 其他的暂时没有)

Beam Stabilizer校准: **CalibrationLaserObj.CalibrationBeamStabilizerObj**

Attenuator校准: **CalibrationLaserObj.CalibrationAttenuatorObj**

暗场自动聚焦校准: `CalibrationLaserObj.CalibrationLaserAutoFocus`

暗场自动聚焦校准: `CalibrationLaserObj.CalibrationLaserOpticalPower`

```
1  /// <summary>
2  /// 激光校准对象
3  /// </summary>
4  [Serializable]
5  public sealed class CalibrationLaserObj
6  {
7      /// <summary>
8      /// 暗场自动聚焦, AB两路灯亮度校准对象
9      /// </summary>
10     public CalibrationLaserAutoFocus CalibrationLaserAutoFocus { get; set; }
    = new CalibrationLaserAutoFocus();
11
12     /// <summary>
13     /// AOD延迟时间校准对象
14     /// </summary>
15     public CalibrationLaserAodDelay CalibrationLaserAodDelay { get; set; }
    = new CalibrationLaserAodDelay();
16
17     /// <summary>
18     /// 暗场相机的像素尺寸校准对象列表
19     /// </summary>
20     public List<CalibrationLaserPixelSizeItem>
    CalibrationLaserPixelSizeItemList { get; set; } = new
    List<CalibrationLaserPixelSizeItem>(0);
21
22     /// <summary>
23     /// Beam Stabilizer校准对象
24     /// </summary>
25     public CalibrationBeamStabilizerObj CalibrationBeamStabilizerObj { get;
    set; } = new CalibrationBeamStabilizerObj();
26
27     /// <summary>
28     /// Attenuator校准对象
29     /// </summary>
30     public CalibrationAttenuatorObj CalibrationAttenuatorObj { get; set; }
    = new CalibrationAttenuatorObj();
31
32     /// <summary>
33     /// 台面功率计校准对象
34     /// </summary>
35     public CalibrationLaserOpticalPower CalibrationLaserOpticalPower { get;
    set; } = new CalibrationLaserOpticalPower();
36 }
```

「 4.1. 自动聚焦校准: `CalibrationLaserAutoFocus` 」

需要使用的属性为以下2个：

CalibrationLaserAutoFocus.CurrentA : A路灯的电流值（绝对电流值），需要下发AF硬件

```
1 public SxExecuteRet<bool> SetSensorCurrentValue(double current, bool
2 isA)
3 {
4     var sxExecuteRet = isA
5         ? Invoke(() => Service!.SetLedA(Convert.ToUInt16(current)))
6         : Invoke(() => Service!.SetLedB(Convert.ToUInt16(current)));
7
8     return sxExecuteRet.IsSuccess == false
9         ? SxExecuteRetHelper.CreateError(sxExecuteRet.Msg, false)
10        : SxExecuteRetHelper.CreateSuccess(true);
11 }
```

CalibrationLaserAutoFocus.CurrentB : B路灯的电流值（绝对电流值），需要下发AF硬件

```
1 public SxExecuteRet<bool> SetSensorCurrentValue(double current, bool
2 isA)
3 {
4     var sxExecuteRet = isA
5         ? Invoke(() => Service!.SetLedA(Convert.ToUInt16(current)))
6         : Invoke(() => Service!.SetLedB(Convert.ToUInt16(current)));
7
8     return sxExecuteRet.IsSuccess == false
9         ? SxExecuteRetHelper.CreateError(sxExecuteRet.Msg, false)
10        : SxExecuteRetHelper.CreateSuccess(true);
11 }
```

```
1 /// <summary>
2 /// 暗场自动聚焦，AB两路灯亮度校准对象
3 /// </summary>
4 [Serializable]
5 public sealed class CalibrationLaserAutoFocus : CalibrationBase
6 {
7     /// <summary>
8     /// A灯电流值
9     /// </summary>
10    public double CurrentA { get; set; }
11
12    /// <summary>
13    /// B灯电流值
14    /// </summary>
15    public double CurrentB { get; set; }
16 }
```

「 4.2. Aod延迟校准: CalibrationLaserAodDelay 」

需要使用的属性为以下1个：

CalibrationLaserAodDelay.AodDelayTime : AOD 延迟时间（绝对延迟时间），需要下发Laser硬件

```

1 public SxExecuteRet<bool> SetSensorAodDelayValue(double aodDelay)
2 {
3     var sxExecuteRet = Invoke(() =>
4         Service!.SetSensorAodDelay(Convert.ToInt32(aodDelay)));
5     return sxExecuteRet.IsSuccess == false
6         ? SxExecuteRetHelper.CreateError(sxExecuteRet.Msg, false)
7         : SxExecuteRetHelper.CreateSuccess(true);
8 }

```

```

1 /// <summary>
2 /// AOD延迟时间校准对象
3 /// </summary>
4 public sealed class CalibrationLaserAodDelay : CalibrationBase
5 {
6     /// <summary>
7     /// AOD 延迟时间
8     /// </summary>
9     public double AodDelayTime { get; set; }
10 }

```

「 4.3. 暗场相机尺寸校准： CalibrationLaserPixelSizeItem 」

需要使用的属性为以下3个：

CalibrationLaserPixelSizeItem.CgMagTypeEnum：当前Mag（cuga中Mag枚举类型，分别对应了低中高）（注意：我们硬件目前只有高Mag，其他的暂时没有）

CalibrationLaserPixelSizeItem.PmtId：通道3 PMT暗场相机ID

CalibrationLaserPixelSizeItem.YPixelSize：当前Mag，当前通道3 PMT暗场相机ID的：y方向1像素转尺寸（um），**Cuga内部使用**

```

1 /// <summary>
2 /// 暗场相机的像素尺寸校准对象
3 /// </summary>
4 [Serializable]
5 public sealed class CalibrationLaserPixelSizeItem : CalibrationBase
6 {
7     /// <summary>
8     /// Mag类型
9     /// </summary>
10    public CgMagTypeEnum CgMagTypeEnum { get; set; }
11
12    /// <summary>
13    /// 暗场相机ID
14    /// </summary>
15    public int PmtId { get; set; }
16
17    /// <summary>
18    /// 单位um/pixel(YSizePerPixel)

```

```

19     /// </summary>
20     public double YPixelSize { get; set; }
21 }

```

「 4.4. 明暗场中心的offset校准： CalibrationLaserLineCentricityItem 」

需要使用的属性为一下3个：

CalibrationLaserLineCentricityItem.CgMagTypeEnum : 当前Mag (cuga中Mag枚举类型, 分别对应了低中高) (注意: 我们硬件目前只有高Mag, 其他的暂时没有)

CalibrationLaserLineCentricityItem.PmtId : 通道3 PMT暗场相机ID

CalibrationLaserLineCentricityItem.OffsetPosition : 当前通道3 PMT暗场相机ID的: 修正后的明暗场中心位置, **Cuga内部使用**

```

1     /// <summary>
2     /// 扫描线中心与明场中心的offset校准
3     /// </summary>
4     [Serializable]
5     public sealed class CalibrationLaserLineCentricityItem : CalibrationBase
6     {
7         /// <summary>
8         /// Mag类型
9         /// </summary>
10        public CgMagTypeEnum CgMagTypeEnum { get; set; }
11
12        /// <summary>
13        /// 暗场相机ID
14        /// </summary>
15        public int PmtId { get; set; }
16
17        /// <summary>
18        /// 校准后的明场机械坐标
19        /// </summary>
20        public double FindBrightMachinePosition { get; set; }
21
22        /// <summary>
23        /// 校准后的暗场机械坐标
24        /// </summary>
25        public double FindDarkMachinePosition { get; set; }
26    }

```

「 4.5. 三个CIB的采样窗口完全同步校准： CalibrationLaserXTCCalibrationItem 」

需要使用的属性为一下5个:

`CalibrationLaserXTCCalibrationItem.CgMagTypeEnum` : 当前Mag (cuga中Mag枚举类型, 分别对应了低中高) (注意: 我们硬件目前只有高Mag, 其他的暂时没有)

`CalibrationLaserXTCCalibrationItem.PmtId` : PMT暗场相机ID, 1到15号光斑

`CalibrationLaserXTCCalibrationItem.Ch1` : Ch1的补偿值

`CalibrationLaserXTCCalibrationItem.Ch2` : Ch2的补偿值

`CalibrationLaserXTCCalibrationItem.Ch3` : Ch3的补偿值

```
1  /// <summary>
2  /// 三个CIB的采样窗口完全同步校准
3  /// </summary>
4  [Serializable]
5  public sealed class CalibrationLaserXTCCalibrationItem : CalibrationBase
6  {
7      /// <summary>
8      /// Mag类型
9      /// </summary>
10     public CgMagTypeEnum CgMagTypeEnum { get; set; }
11
12     /// <summary>
13     /// 暗场相机ID
14     /// </summary>
15     public int PmtId { get; set; }
16
17     /// <summary>
18     /// Ch1补偿值
19     /// </summary>
20     public int CH1 { get; set; }
21
22     /// <summary>
23     /// Ch2补偿值
24     /// </summary>
25     public int CH2 { get; set; }
26
27     /// <summary>
28     /// Ch3补偿值
29     /// </summary>
30     public int CH3 { get; set; }
31 }
```

「 4.6. 使扫描线各处的强度一致校准: `CalibrationIllumination` 」

需要使用的属性为一下5个:

`CalibrationIllumination.CgMagTypeEnum` : 当前Mag (cuga中Mag枚举类型, 分别对应了低中高) (注意: 我们硬件目前只有高Mag, 其他的暂时没有)

`CalibrationIllumination.PmtId` : PMT暗场相机ID, 1到15号光斑

`CalibrationIllumination.ChannelId` : PMT暗场相机ID的通道, 1到3通道

`CalibrationIllumination.RatioValue` : 扫描线于平均值比值


```

1  /// <summary>
2  /// 使扫描线各处的强度一致校准
3  /// </summary>
4  [Serializable]
5  public sealed class CalibrationIllumination : CalibrationBase
6  {
7      /// <summary>
8      /// 波形功率系数(1表示100%, 0表示0%)
9      /// </summary>
10     public double Coefficient { get; set; }
11
12     /// <summary>
13     /// Mag类型
14     /// </summary>
15     public CgMagTypeEnum CgMagTypeEnum { get; set; }
16
17     /// <summary>
18     /// 扫描线与平均值比值
19     /// </summary>
20     public List<double> RatioValue { get; set; }
21 }

```

「 4.7. Beam Stabilizer校准: CalibrationBeamStabilizerObj 」

需要使用以下属性:

CalibrationBeamStabilizerObj.CurrentPDPosition1 : 当前四象限PD1坐标

CalibrationBeamStabilizerObj.CurrentPDPosition2 : 当前四象限PD2坐标

```

1  public sealed class CalibrationBeamStabilizerObj : CalibrationBase
2  {
3      /// <summary>
4      /// 四象限PD1坐标
5      /// </summary>
6      public CgPoint CurrentPDPosition1 { get; set; }
7
8      /// <summary>
9      /// 四象限PD2坐标
10     /// </summary>
11     public CgPoint CurrentPDPosition2 { get; set; }
12 }

```

「 4.8. Attenuator校准: CalibrationAttenuatorObj 」

需要使用以下属性:

`CalibrationAttenuatorObj.CoefficientCurvePositions` : 台面功率与系数C之间的曲线值(C,Pc/P)的一系列List值。

`CalibrationAttenuatorObj.CoefficientFitCurvePositions` : 系数曲线CoefficientCurvePositions通过三次多项式拟合后的曲线值。

```
1 public sealed class CalibrationAttenuatorObj : CalibrationBase
2 {
3     /// <summary>
4     /// 台面功率与系数C之间的曲线值(C,Pc/P)的一系列List值。
5     /// </summary>
6     public List<CgPoint> CoefficientCurvePositions { get; set; }
7
8     /// <summary>
9     /// 系数曲线通过三次多项式拟合后的曲线值
10    /// </summary>
11    public List<CgPoint> CoefficientFitCurvePositions { get; set; }
12 }
```

「 4.9. OpticalPower台面功率计校准: CalibrationLaserOpticalPower 」

需要使用以下属性:

`CalibrationLaserOpticalPower.MeasureMaxPower` : 测量的最大功率

`CalibrationLaserOpticalPower.MeasureMaxPowerPosition` : 测量的最大功率的机械坐标

```
1 /// <summary>
2 /// 台面功率计校准对象
3 /// </summary>
4 [Serializable]
5 public sealed class CalibrationLaserOpticalPower : CalibrationBase
6 {
7     /// <summary>
8     /// 测量的最大功率
9     /// </summary>
10    public double MeasureMaxPower { get; set; }
11
12    /// <summary>
13    /// 测量的最大功率机械坐标
14    /// </summary>
15    public CgPoint MeasureMaxPowerPosition { get; set; }
16 }
```

5. 缓震平台: CalibrationAdsObj

ads一共3个校准:

压力前馈、X方向速度前馈、y方向速度前馈校准: 分别对应一下属性

压力前馈校准: `CalibrationAdsObj.CalibrationAdsPressureGains`

X方向速度前馈校准: `CalibrationAdsObj.CalibrationAdsXGainsItemList` 是列表, 列表中有3个对象, 因为我们有多个速度挡位: 低中高

y方向速度前馈校准: `CalibrationAdsObj.CalibrationAdsYGainsItemList` 是列表, 列表中有1个对象, 只需要做高速挡位

「 5.1. 压力前馈校准: `CalibrationAdsPressureGains` 」

需要使用的属性为以下1个:

`CalibrationAdsPressureGains.PressureValue1` : 比例阀输出值1 (绝对), 需要下发ADS硬件

`CalibrationAdsPressureGains.PressureValue2` : 比例阀输出值2 (绝对), 需要下发ADS硬件

`CalibrationAdsPressureGains.PressureValue3` : 比例阀输出值3 (绝对), 需要下发ADS硬件

```
1 public SxExecuteRet<bool> SetSensorFeedForwardPressureValue(double
   pressureValue1, double pressureValue2, double pressureValue3)
2 {
3     var sxExecuteRet = Invoke(() =>
        Service!.SetSensorPressureValue(Convert.ToUInt16(pressureValue1),
        Convert.ToUInt16(pressureValue2), Convert.ToUInt16(pressureValue3)));
4     return sxExecuteRet.IsSuccess == false
5         ? SxExecuteRetHelper.CreateError(sxExecuteRet.Msg, false)
6         : SxExecuteRetHelper.CreateSuccess(true);
7 }
```

```
1 /// <summary>
2 /// 压力前馈校准对象
3 /// </summary>
4 [Serializable]
5 public sealed class CalibrationAdsPressureGains : CalibrationBase
6 {
7     /// <summary>
8     /// 比例阀输出值1
9     /// </summary>
10    public double PressureValue1 { get; set; }
11
12    /// <summary>
13    /// 比例阀输出值2
14    /// </summary>
15    public double PressureValue2 { get; set; }
16
17    /// <summary>
18    /// 比例阀输出值3
19    /// </summary>
20    public double PressureValue3 { get; set; }
21 }
```

「 5.2. X方向速度前馈校准： CalibrationAdsXGainsItem 」

需要使用的属性为以下1个：

CalibrationAdsXGainsItem.Speed : 当前速度档位 StageSpeedEnum.Low $\Rightarrow$ 0, StageSpeedEnum.Middle $\Rightarrow$ 1, StageSpeedEnum.High $\Rightarrow$ 2

CalibrationAdsXGainsItem.PositiveX1 : X正速度前馈系数X1 (绝对) , 需要下发ADS硬件

CalibrationAdsXGainsItem.PositiveX2 : X正速度前馈系数X2 (绝对) , 需要下发ADS硬件

CalibrationAdsXGainsItem.NegativeX3 : X负速度前馈系数X3 (绝对) , 需要下发ADS硬件

CalibrationAdsXGainsItem.NegativeX4 : X负速度前馈系数X4 (绝对) , 需要下发ADS硬件

```
1 public SxExecuteRet<bool>
   SetSensorXSpeedFeedForwardValue(StageSpeedEnum stageSpeedEnum, bool
   isPositive, (double X1, double X2) value)
2 {
3     var sxExecuteRet = Invoke(()  $\Rightarrow$ 
   Service!.SetXSpeedFeed(stageSpeedEnum.ToAdsSpeedEnum(), isPositive,
   (Convert.ToUInt16(value.X1), Convert.ToUInt16(value.X2)));
4
5     return sxExecuteRet.IsSuccess == false
6         ? SxExecuteRetHelper.CreateError(sxExecuteRet.Msg, false)
7         : SxExecuteRetHelper.CreateSuccess(true);
8 }
```

```
1 /// <summary>
2 /// X方向速度前馈校准对象
3 /// </summary>
4 [Serializable]
5 public sealed class CalibrationAdsXGainsItem : CalibrationBase
6 {
7     /// <summary>
8     /// 速度前馈校准速度
9     /// </summary>
10    public int Speed { get; set; }
11
12    /// <summary>
13    /// X正速度前馈系数X1
14    /// </summary>
15    public double PositiveX1 { get; set; }
16
17    /// <summary>
18    /// X正速度前馈系数X2
19    /// </summary>
20    public double PositiveX2 { get; set; }
21
22    /// <summary>
23    /// X负速度前馈系数X3
24    /// </summary>
```

```

25     public double NegativeX3 { get; set; }
26
27     /// <summary>
28     /// X负速度前馈系数X4
29     /// </summary>
30     public double NegativeX4 { get; set; }
31 }

```

「 5.2. Y方向速度前馈校准： CalibrationAdsYGainsItem 」

需要使用的属性为以下1个：

CalibrationAdsYGainsItem.Speed : 当前速度挡位 StageSpeedEnum.Low $\Rightarrow$ 0, StageSpeedEnum.Middle $\Rightarrow$ 1, StageSpeedEnum.High $\Rightarrow$ 2。 **目前只有高挡位**

CalibrationAdsYGainsItem.PositiveY1 : Y正速度前馈系数Y1 (绝对) , 需要下发ADS硬件

CalibrationAdsYGainsItem.PositiveY2 : Y正速度前馈系数Y2 (绝对) , 需要下发ADS硬件

CalibrationAdsYGainsItem.PositiveY3 : Y正速度前馈系数Y3 (绝对) , 需要下发ADS硬件

CalibrationAdsYGainsItem.NegativeY4 : Y负速度前馈系数Y4 (绝对) , 需要下发ADS硬件

CalibrationAdsYGainsItem.NegativeY5 : Y负速度前馈系数Y5 (绝对) , 需要下发ADS硬件

CalibrationAdsYGainsItem.NegativeY6 : Y负速度前馈系数Y6 (绝对) , 需要下发ADS硬件

```

1  public SxExecuteRet<bool>
   SetSensorYSpeedFeedForwardValue(StageSpeedEnum stageSpeedEnum, bool
   isPositive, (double Y1, double Y2, double Y3) value)
2  {
3      var sxExecuteRet = Invoke(()  $\Rightarrow$ 
   Service!.SetYSpeedFeed(stageSpeedEnum.ToAdsSpeedEnum(), isPositive,
   (Convert.ToUInt16(value.Y1), Convert.ToUInt16(value.Y2),
   Convert.ToUInt16(value.Y3)));
4
5      return sxExecuteRet.IsSuccess == false
6          ? SxExecuteRetHelper.CreateError(sxExecuteRet.Msg, false)
7          : SxExecuteRetHelper.CreateSuccess(true);
8  }

```

```

1  /// <summary>
2  /// Y方向速度前馈校准对象
3  /// </summary>
4  [Serializable]
5  public sealed class CalibrationAdsYGainsItem : CalibrationBase
6  {
7      /// <summary>
8      /// 速度前馈校准速度
9      /// </summary>
10     public int Speed { get; set; }
11
12     /// <summary>

```

```
13    /// Y正速度前馈系数Y1
14    /// </summary>
15    public double PositiveY1 { get; set; }
16
17    /// <summary>
18    /// Y正速度前馈系数Y2
19    /// </summary>
20    public double PositiveY2 { get; set; }
21
22    /// <summary>
23    /// Y正速度前馈系数Y3
24    /// </summary>
25    public double PositiveY3 { get; set; }
26
27    /// <summary>
28    /// Y负速度前馈系数Y4
29    /// </summary>
30    public double NegativeY4 { get; set; }
31
32    /// <summary>
33    /// Y负速度前馈系数Y5
34    /// </summary>
35    public double NegativeY5 { get; set; }
36
37    /// <summary>
38    /// Y负速度前馈系数Y6
39    /// </summary>
40    public double NegativeY6 { get; set; }
41 }
```

##